ABHIJATRI — COMPLETE TEST CASE & SECURITY CHECKLIST

For the Review Lifecycle (Current Implementation Scope)

Contents

1 FUNCTIONAL TEST CASES — API ENDPOINTS **2**
- 1.1 Health Checks . . . 2
- 1.2 Guest Review Submission (POST /api/v1/reviews/guest) . . . 2
- 1.3 Public Review Feed (GET /destinations/{id}/reviews) . . . 3
- 1.4 Receipt Exchange (POST /api/v1/review-receipts/exchange) . . . 3
- 1.5 Withdrawal (POST /api/v1/reviews/{review_id}/withdraw) . . . 3
- 1.6 Staff Moderation Queue (GET /api/v1/staff/moderation/reviews) . . . 4
- 1.7 Moderation Decisions . . . 4
- 1.8 Conflict Flag . . . 4

2 SECURITY TEST CASES **5**
- 2.1 Injection Attacks . . . 5
- 2.2 XSS (Cross-Site Scripting) . . . 5
- 2.3 IDOR (Insecure Direct Object Reference) . . . 5
- 2.4 CSRF (Cross-Site Request Forgery) . . . 5
- 2.5 Receipt/Token Security . . . 6
- 2.6 Header & Method Security . . . 6

3 DATABASE INTEGRITY & CONCURRENCY TESTS **6**
- 3.1 Concurrency / Race Conditions . . . 7
- 3.2 Row-Level Security . . . 7

4 INPUT VALIDATION & EDGE CASES **7**
- 4.1 Bangla-Specific Validation . . . 7

5 AUTHENTICATION & AUTHORIZATION TESTS **8**

6 PRIVACY & DATA GOVERNANCE TESTS **8**

7 PERFORMANCE & ABUSE PREVENTION TESTS **8**
- 7.1 Load Testing . . . 9

8 OFFLINE & CACHE SAFETY TESTS **9**

9 CONFIGURATION & DEPLOYMENT SAFETY TESTS **9**
- 9.1 CI Pipeline Checks . . . 9

10 MANUAL WALKTHROUGH CHECKLIST **10**

11 PRIORITY MATRIX **10**

1 FUNCTIONAL TEST CASES — API ENDPOINTS

1.1 Health Checks

- **TC-F01** GET `/api/v1/health` returns 200 with `{"status": "ok"}`
- **TC-F02** HEAD `/api/v1/health` returns 200 with no body
- **TC-F03** GET `/api/v1/health/ready` returns 200 when DB is up
- **TC-F04** GET `/api/v1/health/ready` returns 503 when DB is down
- **TC-F05** HEAD `/api/v1/health/ready` returns 200 when DB is up

1.2 Guest Review Submission (POST /api/v1/reviews/guest)

- **TC-F10** Submit a valid guest review → 201, returns `review_id`, `status=PENDING`, `version=1`, `receipt_token` (non-empty), `duplicate_recovered=false`
- **TC-F11** Retry with same `client_draft_id` + same `guest_scope_secret` → returns the same `review_id` with `duplicate_recovered=true`
- **TC-F12** Retry with same `client_draft_id` + DIFFERENT `guest_scope_secret` → creates a NEW review (different scope = different guest)
- **TC-F13** Submit to a non-existent `destination_id` → 404
- **TC-F14** Submit to a DRAFT (non-published) destination → 404
- **TC-F15** Submit with `overall_rating` = 0 → 422
- **TC-F16** Submit with `overall_rating` = 6 → 422
- **TC-F17** Submit with body shorter than 20 characters → 422
- **TC-F18** Submit with body longer than 6000 characters → 422
- **TC-F19** Submit with title shorter than 3 characters → 422
- **TC-F20** Submit with title longer than 160 characters → 422
- **TC-F21** Submit with `guest_scope_secret` shorter than 43 characters → 422
- **TC-F22** Submit with `visit_month` in the future → 422
- **TC-F23** Submit with `visit_month` day != 1 (e.g. 2026-09-15) → 422
- **TC-F24** Submit with `guest_display_name` = "Admin" → 409 (reserved name)
- **TC-F25** Submit with `guest_display_name` = "Moderator" → 409
- **TC-F26** Submit with `guest_display_name` containing "" → 409
- **TC-F27** Submit with `guest_display_name` that is NFC-normalizable Bangla → succeeds, stored as NFC-normalized
- **TC-F28** Submit with empty `guest_display_name` → 422
- **TC-F29** Submit with missing required fields → 422 for each missing field
- **TC-F30** Submit with invalid UUID for `destination_id` → 422

- **TC-F31** Submit with invalid enum value for `language` → 422
- **TC-F32** Submit with invalid enum for `traveller_type` → 422
- **TC-F33** Submit with invalid enum for `comfort_tier` → 422

1.3 Public Review Feed (GET /destinations/{id}/reviews)

- **TC-F40** Fetch reviews for a destination with published reviews → 200, items contain only PUBLISHED reviews
- **TC-F41** Fetch reviews → items do NOT contain PENDING, HIDDEN, REJECTED, WITHDRAWN, or DELETED reviews
- **TC-F42** Fetch with `limit=1` → returns 1 item + `next_cursor` (if more exist)
- **TC-F43** Paginate with `?after=<cursor>` → returns the next page correctly
- **TC-F44** Fetch with invalid cursor → 422
- **TC-F45** Fetch with `limit=0` → 422
- **TC-F46** Fetch with `limit=51` → 422
- **TC-F47** Fetch with `limit=50` → succeeds
- **TC-F48** Fetch for a destination with zero published reviews → 200, empty items, `next_cursor=null`
- **TC-F49** `guest_rating_average` and `guest_rating_count` reflect only counted Guest ratings
- **TC-F50** Verify ordering is `published_at` DESC, `id` DESC

1.4 Receipt Exchange (POST /api/v1/review-receipts/exchange)

- **TC-F60** Exchange a valid `receipt_token` → 200, returns correct review status
- **TC-F61** Exchange an invalid/random `receipt_token` → 404
- **TC-F62** Exchange a revoked receipt → 404
- **TC-F63** Exchange with wrong `review_id` + valid token → 404
- **TC-F64** Exchange with `receipt_token` shorter than 40 chars → 422

1.5 Withdrawal (POST /api/v1/reviews/{review_id}/withdraw)

- **TC-F70** Withdraw a PENDING review → status becomes WITHDRAWN, version increments
- **TC-F71** Withdraw a PUBLISHED review → status WITHDRAWN, `counted_rating` removed from aggregate
- **TC-F72** Withdraw an already WITHDRAWN review → idempotent, returns current
- **TC-F73** Withdraw with stale `expected_version` → 409
- **TC-F74** Withdraw with wrong `receipt_token` → 404
- **TC-F75** Verify aggregate stats decrease correctly after withdrawal

1.6 Staff Moderation Queue (GET /api/v1/staff/moderation/reviews)

- **TC-F80** Staff user sees PENDING and UNDER_REVIEW reviews, sorted by `created_at` ASC
- **TC-F81** Returns at most 100 items
- **TC-F82** Non-staff user (no role) → 403
- **TC-F83** No `X-Test-Staff-Id` header → 401
- **TC-F84** Invalid UUID in `X-Test-Staff-Id` → 401
- **TC-F85** In production mode (`APP_ENV` != development/test) → 501

1.7 Moderation Decisions

- **TC-F90** PUBLISH a PENDING review → `status=PUBLISHED`, version increments, `published_at` set
- **TC-F91** PUBLISH an UNDER_REVIEW review → same result
- **TC-F92** PUBLISH a PUBLISHED review → 409 (invalid transition)
- **TC-F93** REJECT a PENDING review → `status=REJECTED`
- **TC-F94** REJECT a PUBLISHED review → 409
- **TC-F95** HIDE a PUBLISHED review → `status=HIDDEN`, `counted_rating` removed
- **TC-F96** HIDE a PENDING review → 409
- **TC-F97** RESTORE a HIDDEN review → `status=PUBLISHED`, rating re-added
- **TC-F98** RESTORE a PENDING review → 409
- **TC-F99** Moderator tries to moderate their own review → 403
- **TC-F100** Moderation with stale `expected_version` → 409
- **TC-F101** Unsupported decision string → 422
- **TC-F102** `reason_code` shorter than 2 chars → 422
- **TC-F103** `reason_code` longer than 60 chars → 422

1.8 Conflict Flag

- **TC-F110** Set conflict on a published review → `conflict_declared=true`, rating changes to INELIGIBLE_CONFLICT
- **TC-F111** Clear conflict → rating recalculated
- **TC-F112** Set conflict with stale version → 409
- **TC-F113** Moderator cannot set conflict on own review → 403
- **TC-F114** Setting conflict to the same value → no-op, returns current state

2 SECURITY TEST CASES

2.1 Injection Attacks

- **TC-S01** SQL Injection in `guest_display_name`: Send `'; DROP TABLE reviews; -` → Expected: 200 or 409 (treated as plain text), no SQL executed
- **TC-S02** SQL Injection in review body: Send `' OR '1'='1` → Expected: stored as literal text, not interpreted
- **TC-S03** SQL Injection in cursor parameter (`?after=`): Send `' UNION SELECT * FROM users-` → Expected: 422 invalid cursor
- **TC-S04** SQL Injection in `reason_code`: Send `'; DELETE FROM moderation_actions;-` → Expected: treated as string
- **TC-S05** NoSQL/JSON Injection in body: Send `{"$gt": ""}` as body value → Expected: 422 (type validation)

2.2 XSS (Cross-Site Scripting)

- **TC-S10** Store XSS in `guest_display_name`: Send `<script>alert('xss')</script>` → Expected: stored as plain text, never rendered as HTML
- **TC-S11** Store XSS in review title: Send `<img src=x onerror=alert(1)>` → Expected: stored as plain text
- **TC-S12** Store XSS in review body: Send `<svg onload=alert(document.cookie)>` → Expected: stored as plain text
- **TC-S13** Verify public feed response Content-Type is `application/json`, not `text/html`

2.3 IDOR (Insecure Direct Object Reference)

- **TC-S20** Try to exchange a receipt for a review that belongs to another guest → 404 (no existence leak)
- **TC-S21** Try to withdraw a review using a receipt from a different review → 404
- **TC-S22** Access staff moderation queue as a non-staff user → 403
- **TC-S23** Access staff moderation queue without authentication header → 401
- **TC-S24** Guess a review UUID and try to moderate it without staff role → 403
- **TC-S25** Verify error messages for 404 do not reveal whether the resource exists

2.4 CSRF (Cross-Site Request Forgery)

- **TC-S30** [FUTURE] When cookie sessions are added: POST `/reviews/guest` without CSRF token → 403
- **TC-S31** [FUTURE] POST `/staff/moderation` with forged origin → blocked
- **TC-S32** Currently: verify all state-changing operations are POST/PUT (not GET)

2.5 Receipt/Token Security

- **TC-S40** Receipt token contains ≥ 256 bits of entropy
- **TC-S41** Server stores only HMAC-SHA-256 digest, not raw token
- **TC-S42** Receipt comparison is constant-time (`hmac.compare_digest`)
- **TC-S43** Brute-force a receipt: try 1000 random tokens → all return 404
- **TC-S44** Receipt exchange is POST, never GET
- **TC-S45** After withdrawal, receipt still returns the review status

2.6 Header & Method Security

- **TC-S50** Send TRACE `/api/v1/health` → 405 Method Not Allowed
- **TC-S51** Send CONNECT `/api/v1/health` → 405
- **TC-S52** Send OPTIONS `/api/v1/reviews/guest` → returns CORS headers
- **TC-S53** GET `/api/v1/reviews/guest` → 405
- **TC-S54** Send oversized Content-Length header → rejected
- **TC-S55** Send request with both Content-Length and Transfer-Encoding → rejected
- **TC-S56** Verify `X-Content-Type-Options: nosniff` on responses
- **TC-S57** Verify `Referrer-Policy` header on responses
- **TC-S58** Verify no `Server` header leaking framework version

3 DATABASE INTEGRITY & CONCURRENCY TESTS

- **TC-D01** `overall_rating` < 1 in DB → CHECK constraint fails
- **TC-D02** `overall_rating` > 5 in DB → CHECK constraint fails
- **TC-D03** `counted_rating` outside 1-5 → CHECK fails
- **TC-D04** `community_rating_count` < 0 → CHECK fails
- **TC-D05** `guest_rating_count` < 0 → CHECK fails
- **TC-D06** Review with BOTH `author_user_id` AND `guest_display_name` → CHECK fails
- **TC-D07** Review with NEITHER `author_user_id` NOR `guest_display_name` → CHECK fails
- **TC-D08** Duplicate (`guest_scope_hash`, `client_draft_id`) → unique index rejects
- **TC-D09** Duplicate (`author_user_id`, `client_draft_id`) → unique index rejects
- **TC-D10** Duplicate active (member, destination, `visit_month`) → unique index rejects

3.1 Concurrency / Race Conditions

- **TC-D20** Two simultaneous PUBLISH decisions → only one succeeds
- **TC-D21** Simultaneous PUBLISH and WITHDRAW → one wins
- **TC-D22** Rapid duplicate guest submissions → only one review created
- **TC-D23** Random sequence of states → run `full_guest_recompute` matches
- **TC-D24** Version counter increments correctly

3.2 Row-Level Security

- **TC-D30** RLS is enabled on every table in the public schema
- **TC-D31** anon and authenticated Supabase roles have no direct grants
- **TC-D32** Data API roles cannot read/write through Supabase Data API

4 INPUT VALIDATION & EDGE CASES

- **TC-V01** Send request body > 1MB → rejected
- **TC-V02** Send deeply nested JSON → rejected or safely handled
- **TC-V03** Send array where object expected → 422
- **TC-V04** Send null for required fields → 422
- **TC-V05** Send integer where string expected → 422
- **TC-V06** Send string > 6000 chars → 422
- **TC-V07** Send Unicode zero-width characters in display name → accepted
- **TC-V08** Send RTL override characters → NFC handles safely
- **TC-V09** Send emoji in display name → accepted
- **TC-V10** Send null bytes (\x00) → rejected
- **TC-V11** Send review with `visit_month` = "not-a-date" → 422
- **TC-V12** Send extra unexpected fields → ignored
- **TC-V13** Send empty JSON body {} → 422
- **TC-V14** Send non-JSON content-type → 422 or 415

4.1 Bangla-Specific Validation

- **TC-V20** Bangla display name requiring NFC → normalizes correctly
- **TC-V21** Bangla conjunct characters → accepted
- **TC-V22** Mixed Bangla + English display name → accepted
- **TC-V23** Reserved name check ignores spaces ("A d m i n" → matches "admin")
- **TC-V24** Bangla equivalent of "Admin" → check reserved list

5 AUTHENTICATION & AUTHORIZATION TESTS

- **TC-A01** Staff endpoint without `X-Test-Staff-Id` header → 401
- **TC-A02** Staff endpoint with non-UUID value → 401
- **TC-A03** Staff endpoint with valid UUID but no staff role → 403
- **TC-A04** Staff endpoint with expired role → 403
- **TC-A05** Staff endpoint with MODERATOR role → 200
- **TC-A06** Staff endpoint with ADMINISTRATOR role → 200
- **TC-A07** Staff endpoint with SUPER_ADMINISTRATOR role → 200
- **TC-A08** `APP_ENV=production`: staff endpoint → 501
- **TC-A09** `APP_ENV=staging`: staff endpoint → 501
- **TC-A10** `X-Test-Staff-Id` accepted ONLY in dev/test
- **TC-A11** Moderator cannot moderate their own review → 403
- **TC-A12** Moderator cannot set conflict on their own review → 403

6 PRIVACY & DATA GOVERNANCE TESTS

- **TC-P01** Public feed does NOT expose `guest_scope_hash`
- **TC-P02** Public feed does NOT expose `receipt_token` or `token_hash`
- **TC-P03** Public feed does NOT expose `author_user_id`
- **TC-P04** Public feed does NOT expose `moderation_actions`
- **TC-P05** Public feed does NOT expose version history
- **TC-P06** Error messages do not reveal DB details/stack traces
- **TC-P07** 404 errors do not differentiate between "doesn't exist" vs "don't have access"
- **TC-P08** Receipt exchange does NOT include body/title
- **TC-P09** Created response does NOT include `guest_scope_hash`
- **TC-P10** No raw IP addresses in JSON responses

7 PERFORMANCE & ABUSE PREVENTION TESTS

- **TC-R01** Rate limit: 50 rapid submissions → throttled
- **TC-R02** Rate limit: 100 receipt exchanges/min → throttled
- **TC-R03** Rate limit on staff endpoints
- **TC-R04** Moderation queue limit=100
- **TC-R05** Public feed pagination prevents full table scan
- **TC-R06** Database connection pool bounded
- **TC-R07** `guest_scope_secret` min_length=43

7.1 Load Testing

- **TC-R10** 50 concurrent submissions → all succeed or fail gracefully
- **TC-R11** 100 concurrent public feed requests → all succeed
- **TC-R12** 20 concurrent moderation decisions → all succeed

8 OFFLINE & CACHE SAFETY TESTS

- **TC-C01** Verify `Cache-Control: no-store` on POST
- **TC-C02** Verify `Cache-Control: no-store` on staff endpoints
- **TC-C03** Verify `Cache-Control: no-store` on receipt exchange
- **TC-C04** Public feed purges on moderation/withdrawal
- **TC-C05** Verify no `Set-Cookie` in cacheable responses
- **TC-C06** Verify GET with request body → rejected
- **TC-C07** Path traversal URL → 404/rejected
- **TC-C08** Encoded slashes URL (%2F) → rejected

9 CONFIGURATION & DEPLOYMENT SAFETY TESTS

- **TC-CFG01** `APP_ENV=production` + default `receipt_pepper` → crashes
- **TC-CFG02** `APP_ENV=production` + short `receipt_pepper` → crashes
- **TC-CFG03** `APP_ENV=production` + `CORS="*"` → crashes
- **TC-CFG04** `APP_ENV=production` + `CORS=""` → crashes
- **TC-CFG05** `APP_ENV=production` + DB without `sslmode=require` → crashes
- **TC-CFG06** `APP_ENV=development` + default pepper → starts
- **TC-CFG07** Interactive docs disabled in production
- **TC-CFG08** Interactive docs enabled in development
- **TC-CFG09** `.env.example` contains no secrets
- **TC-CFG10** `.gitignore` includes `.env`

9.1 CI Pipeline Checks

- **TC-CI01** CI runs lint (ruff/flake8/mypy)
- **TC-CI02** CI runs unit tests
- **TC-CI03** CI runs integration tests against PostgreSQL
- **TC-CI04** CI runs schema drift check (`test_docs_schema.py`)
- **TC-CI05** CI runs OpenAPI test (`test_openapi_contract.py`)

- **TC-CI06** CI runs dependency audit
- **TC-CI07** CI runs secret scan
- **TC-CI08** RLS test (`test_rls.py`) passes in CI

10 MANUAL WALKTHROUGH CHECKLIST

1. Submit a valid guest review. Save the `receipt_token`.
2. Submit exact same review. Verify `duplicate_recovered=true`.
3. Check public feed — review should NOT appear.
4. View moderation queue. Verify review exists.
5. PUBLISH the review.
6. Check public feed — review SHOULD appear.
7. Verify rating average and count updated.
8. HIDE the review.
9. Check public feed — review DISAPPEARS.
10. RESTORE the review.
11. Use `receipt_token` to exchange — verify status.
12. Use `receipt_token` to WITHDRAW.
13. Try to PUBLISH a withdrawn review → 409.

11 PRIORITY MATRIX

Priority	Test Cases
CRITICAL	TC-CFG01 to TC-CFG05, TC-S40 to TC-S44, TC-D01 to TC-D10, TC-A08 to TC-A10, TC-P01 to TC-P10
HIGH	TC-S01 to TC-S05, TC-S10 to TC-S13, TC-S20 to TC-S25, TC-D20 to TC-D24, TC-C01 to TC-C08
MEDIUM	TC-R01 to TC-R07, TC-S50 to TC-S58, TC-R10 to TC-R12
LOW	TC-V07 to TC-V14, TC-V20 to TC-V24